

15-WEEK COMPANION HANDBOOK

A professional reading guide and decision framework spanning Python, data science, machine learning, deep learning, deployment, and generative AI.

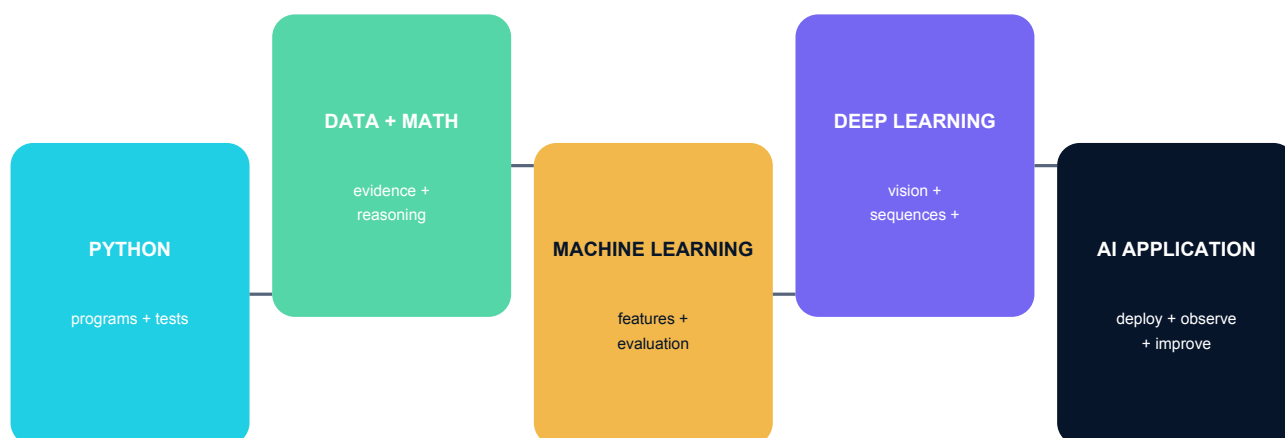


15 WEEKS	125 LECTURES	FULL STACK
15 weekly guides	Professional resource edition	Learn + build + explain

LEARN THE WHOLE PATH

A full-stack AI engineer can move from an ambiguous problem to a measured, deployable system. That path crosses programming, data, mathematics, models, evaluation, software interfaces, operations, and generative AI application design.

The goal is not to memorize every API. It is to recognize the layer you are working in, preserve contracts between layers, and produce evidence that the complete system works.



THE ENGINEERING LOOP

1	FRAME	Define the user, task, data boundary, risk, quality metric, and operating constraint.
2	BASELINE	Create the simplest correct solution and a reproducible evaluation path.
3	LEARN	Improve data, features, architecture, optimization, or context from observed errors.
4	SHIP	Package a versioned artifact behind a tested interface with safe configuration.
5	OPERATE	Monitor quality, latency, failures, security, cost, and feedback; then iterate.

LEARNING CONTRACT

Use these rules in every notebook, script, model, and application.

- Create a clean environment and record interpreter, library, data, and model versions.
- Separate training, validation, and test responsibilities; prevent preprocessing and target leakage.
- Start with a baseline and select metrics from the cost of real errors, not from convenience.
- Make tensor and table contracts explicit: shape, dtype, schema, units, missingness, and allowed values.
- Test invalid inputs and failure behavior in addition to the happy path.
- Keep credentials outside code and notebooks; validate provider output before using it downstream.
- Finish every project with a README, reproducible command, evaluation table, limitations, and next experiment.

THE SEVEN NUMBERS TO CARRY

DATA	rows / classes / missingness	What evidence is available and where is it weak?
QUALITY	task metric + segment metric	Does the system solve the intended task?
GENERALIZATION	train vs validation vs test	Will the result survive unseen data?
CALIBRATION	probability vs observed rate	Can confidence support a decision?
LATENCY	p50 / p95 / p99	How responsive is the deployed path?
RELIABILITY	errors + invalid-output rate	How does the system fail?
UNIT COST	cost per successful outcome	Is the complete path efficient?

Python Programming Basics

This week advances the full-stack path through python programming basics. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

006	Introduction to Week 1 Python Programming Basics
007	Day 1: Introduction to Python and Development Setup
008	Day 2: Control Flow in Python
009	Day 3: Functions and Modules
010	Day 4: Data Structures (Lists, Tuples, Dictionaries, Sets)
011	Day 5: Working with Strings
012	Day 6: File Handling
013	Day 7: Pythonic Code and Project Work

WEEKLY LENS

Write small, readable programs and make the environment reproducible before optimizing anything.

Evidence standard

Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Working loop

```
python -m venv .venv && python -m  
pytest
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Data Science Essentials

This week advances the full-stack path through data science essentials. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

014	Introduction to Week 2 Data Science Essentials
015	Day 1: Introduction to NumPy for Numerical Computing
016	Day 2: Advanced NumPy Operations
017	Day 3: Introduction to Pandas for Data Manipulation
018	Day 4: Data Cleaning and Preparation with Pandas
019	Day 5: Data Aggregation and Grouping in Pandas
020	Day 6: Data Visualization with Matplotlib and Seaborn
021	Day 7: Exploratory Data Analysis (EDA) Project

WEEKLY LENS

Treat data work as a traceable pipeline: inspect shape and types, clean deliberately, visualize honestly, and preserve assumptions.

Evidence standard

State inputs, assumptions, output contract, metric, failure case, version information, and the next decision.

Working loop

```
define -> build -> test -> measure ->
explain
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Mathematics for Machine Learning

This week advances the full-stack path through mathematics for machine learning. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

022	Introduction to Week 3 Mathematics for Machine Learning
023	Day 1: Linear Algebra Fundamentals
024	Day 2: Advanced Linear Algebra Concepts
025	Day 3: Calculus for Machine Learning (Derivatives)
026	Day 4: Calculus for Machine Learning (Integrals and Optimization)
027	Day 5: Probability Theory and Distributions
028	Day 6: Statistics Fundamentals
029	Day 7: Math-Driven Mini Project - Linear Regression from Scratch

WEEKLY LENS

Use mathematics to explain what a model computes, what an optimizer changes, and why the result should be trusted.

Evidence standard

State inputs, assumptions, output contract, metric, failure case, version information, and the next decision.

Working loop

```
define -> build -> test -> measure ->
explain
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Probability and Statistics for Machine Learning

This week advances the full-stack path through probability and statistics for machine learning. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

030	Introduction to Week 4 Probability and Statistics for Machine Learning
031	Day 1: Probability Theory and Random Variables
032	Day 2: Probability Distributions in Machine Learning
033	Day 3: Statistical Inference - Estimation and Confidence Intervals
034	Day 4: Hypothesis Testing and P-Values
035	Day 5: Types of Hypothesis Tests
036	Day 6: Correlation and Regression Analysis
037	Day 7: Statistical Analysis Project - Analyzing Real-World Data

WEEKLY LENS

Reason from uncertainty: define the population, sampling process, assumptions, effect size, and decision before reading a p-value.

Evidence standard

Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Working loop

```
question -> estimand -> assumptions ->
interval -> decision
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Introduction to Machine Learning

This week advances the full-stack path through introduction to machine learning. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

038	Introduction to Week 5 Introduction to Machine Learning
039	Day 1: Machine Learning Basics and Terminology
040	Day 2: Introduction to Supervised Learning and Regression Models
041	Day 3: Advanced Regression Models - Polynomial Regression and Regularization
042	Day 4: Introduction to Classification and Logistic Regression
043	Day 5: Model Evaluation and Cross-Validation
044	Day 6: k-Nearest Neighbors (k-NN) Algorithm
045	Day 7: Supervised Learning Mini Project

WEEKLY LENS

Establish a leakage-safe baseline, choose a model family from the task, and evaluate against a metric tied to the real error cost.

Evidence standard

State inputs, assumptions, output contract, metric, failure case, version information, and the next decision.

Working loop

```
define -> build -> test -> measure ->
explain
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Feature Engineering and Model Evaluation

This week advances the full-stack path through feature engineering and model evaluation. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

046	Introduction to Week 6 Feature Engineering and Model Evaluation
047	Day 1: Introduction to Feature Engineering
048	Day 2: Data Scaling and Normalization
049	Day 3: Encoding Categorical Variables
050	Day 4: Feature Selection Techniques
051	Day 5: Creating and Transforming Features
052	Day 6: Model Evaluation Techniques
053	Day 7: Cross-Validation and Hyperparameter Tuning

WEEKLY LENS

Fit every transformation on training data only, keep preprocessing inside the pipeline, and test that features will exist at inference.

Evidence standard

Fit preprocessing on training folds only; test unseen categories, missing values, feature order, and schema at prediction time.

Working loop

```
Pipeline([('prep', transformer),
          ('model', estimator)])
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Advanced Machine Learning Algorithms

This week advances the full-stack path through advanced machine learning algorithms. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

054	Introduction to Week 7 Advanced Machine Learning Algorithms
055	Day 1: Introduction to Ensemble Learning
056	Day 2: Bagging and Random Forests
057	Day 3: Boosting and Gradient Boosting
058	Day 4: Introduction to XGBoost
059	Day 5: LightGBM and CatBoost
060	Day 6: Handling Imbalanced Data
061	Day 7: Ensemble Learning Project - Comparing Models on a Real Dataset

WEEKLY LENS

Ensembles trade simplicity for predictive strength; compare them on honest validation data, operational cost, and calibration.

Evidence standard

State inputs, assumptions, output contract, metric, failure case, version information, and the next decision.

Working loop

```
define -> build -> test -> measure ->
explain
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Model Tuning and Optimization

This week advances the full-stack path through model tuning and optimization. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

062	Introduction to Week 8 Model Tuning and Optimization
063	Day 1: Introduction to Hyperparameter Tuning
064	Day 2: Grid Search and Random Search
065	Day 3: Advanced Hyperparameter Tuning with Bayesian Optimization
066	Day 4: Regularization Techniques for Model Optimization
067	Day 5: Cross-Validation and Model Evaluation Techniques
068	Day 6: Automated Hyperparameter Tuning with GridSearchCV and RandomizedSearchCV
069	Day 7: Optimization Project - Building and Tuning a Final Model

WEEKLY LENS

Tune the search process, not just the model: define the space, budget, objective, cross-validation scheme, and stopping rule.

Evidence standard

Record distributions, folds, seed, trials, failures, runtime, best score uncertainty, and untouched final test performance.

Working loop

```
search.fit(X_train, y_train);
search.best_params_
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Neural Networks and Deep Learning Fundamentals

This week advances the full-stack path through neural networks and deep learning fundamentals. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

070	Introduction to Week 9 Neural Networks and Deep Learning Fundamentals
071	Day 1: Introduction to Deep Learning and Neural Networks
072	Day 2: Forward Propagation and Activation Functions
073	Day 3: Loss Functions and Backpropagation
074	Day 4: Gradient Descent and Optimization Techniques
075	Day 5: Building Neural Networks with TensorFlow and Keras
076	Day 6: Building Neural Networks with PyTorch
077	Day 7: Neural Network Project - Image Classification on CIFAR-10

WEEKLY LENS

A neural network is a differentiable program; understand tensor shapes, forward computation, loss, gradients, and optimizer state.

Evidence standard

Assert tensor shapes, overfit a tiny batch, inspect loss and gradients, separate train/eval behavior, and save the best checkpoint.

Working loop

```
zero_grad -> forward -> loss ->
backward -> step
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Convolutional Neural Networks (CNNs)

This week advances the full-stack path through convolutional neural networks (cnns). Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

078	Introduction to Week 10 Convolutional Neural Networks (CNNs)
079	Day 1: Introduction to Convolutional Neural Networks
080	Day 2: Convolutional Layers and Filters
081	Day 3: Pooling Layers and Dimensionality Reduction
082	Day 4: Building CNN Architectures with Keras and TensorFlow
083	Day 5: Building CNN Architectures with PyTorch
084	Day 6: Regularization and Data Augmentation for CNNs
085	Day 7: CNN Project - Image Classification on Fashion MNIST or CIFAR-10

WEEKLY LENS

CNNs exploit spatial structure. Track receptive field, feature-map shape, augmentation policy, and performance by class.

Evidence standard

Trace tensor shapes and receptive field; visualize errors by class; report augmentation, normalization, latency, and parameter count.

Working loop

```
N,C,H,W -> conv -> activation -> pool
-> head
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Recurrent Neural Networks (RNNs) and Sequence Modeling

This week advances the full-stack path through recurrent neural networks (rnns) and sequence modeling. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

086	Introduction to Week 11 Recurrent Neural Networks (RNNs) and Sequence Modeling
087	Day 1: Introduction to Sequence Modeling and RNNs
088	Day 2: Understanding RNN Architecture and Backpropagation Through Time (BPTT)
089	Day 3: Long Short-Term Memory (LSTM) Networks
090	Day 4: Gated Recurrent Units (GRUs)
091	Day 5: Text Preprocessing and Word Embeddings for RNNs
092	Day 6: Sequence-to-Sequence Models and Applications
093	Day 7: RNN Project - Text Generation or Sentiment Analysis

WEEKLY LENS

Sequence models carry state through time; manage ordering, padding, truncation, hidden state, and gradient stability explicitly.

Evidence standard

Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Working loop

```
tokens -> embeddings -> recurrent
state -> task head
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Transformers and Attention Mechanisms

This week advances the full-stack path through transformers and attention mechanisms. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

094	Introduction to Week 12 Transformers and Attention Mechanisms
095	Day 1: Introduction to Attention Mechanisms
096	Day 2: Introduction to Transformers Architecture
097	Day 3: Self-Attention and Multi-Head Attention in Transformers
098	Day 4: Positional Encoding and Feed-Forward Networks
099	Day 5: Hands-On with Pre-Trained Transformers - BERT and GPT
100	Day 6: Advanced Transformers - BERT Variants and GPT-3
101	Day 7: Transformer Project - Text Summarization or Translation

WEEKLY LENS

Transformers mix token information with attention and positional signals; evaluate both task quality and computational cost.

Evidence standard

Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Working loop

$QK^T/\sqrt{d} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow V$

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Transfer Learning and Fine-Tuning

This week advances the full-stack path through transfer learning and fine-tuning. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

102	Introduction to Week 13 Transfer Learning and Fine-Tuning
103	Day 1: Introduction to Transfer Learning
104	Day 2: Transfer Learning in Computer Vision
105	Day 3: Fine-Tuning Techniques in Computer Vision
106	Day 4: Transfer Learning in NLP
107	Day 5: Fine-Tuning Techniques in NLP
108	Day 6: Domain Adaptation and Transfer Learning Challenges
109	Day 7: Transfer Learning Project - Fine-Tuning for a Custom Task

WEEKLY LENS

Transfer learning starts from useful representations; control which parameters move, use a smaller learning rate, and watch for forgetting.

Evidence standard

Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Working loop

```
load -> freeze -> train head ->
unfreeze selectively -> validate
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

MLOps and Model Deployment

This week advances the full-stack path through mlops and model deployment. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

110	Introduction to Week 14 MLOps and Model Deployment
111	Day 1: Introduction to MLOps - Why It Matters
112	Day 2: Version Control for Data and Models (Git, DVC)
113	Day 3: Model Packaging with Pickle, ONNX, and TorchScript
114	Day 4: Serving Models with Flask, FastAPI, and Streamlit
115	Day 5: Deploying ML Models on AWS, GCP, and Azure
116	Day 6: CI/CD Pipelines for Machine Learning Projects
117	Day 7: MLOps Project - Build and Deploy a Model End-to-End

WEEKLY LENS

Production ML connects versioned inputs to a deployable artifact, tested interface, observable service, and reversible release.

Evidence standard

Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Working loop

```
build -> test -> package -> deploy ->
verify -> observe -> rollback
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

Generative AI and Large Language Model Applications

This week advances the full-stack path through generative ai and large language model applications. Learn the mechanism, implement a bounded example, and finish with evidence another engineer can reproduce.

118	Introduction to Week 15 - Generative AI and Large Language Model Applications
119	Day 1: Introduction to Generative AI and LLMs
120	Day 2: Understanding Prompt Engineering and Tokenization
121	Day 3: Building Applications with OpenAI, Gemini, and Claude APIs
122	Day 4: Fine-Tuning and Custom Instruction Models
123	Day 5: Building RAG (Retrieval-Augmented Generation) Pipelines
124	Day 6: AI Agents and Autonomous Systems Overview
125	Day 7: Generative AI Project - Build Your Own Chatbot or Content Generator

WEEKLY LENS

Generative AI applications are systems, not prompts: provider boundary, context, retrieval, tools, evaluation, safety, latency, and cost all matter.

Evidence standard

Keep keys server-side; pin provider/model metadata; add timeout, retry, rate limits, schema checks, logging, evaluations, and cost limits.

Working loop

```
request -> provider adapter ->  
validate -> evaluate -> return
```

Exit ticket

Show the artifact, one measurement, one failure case, and the decision the result changes.

READ - BUILD - TEST - EXPLAIN

Read for the enduring idea. Build a thin proof. Test both expected and failure behavior. Explain assumptions, limits, measurements, and the next decision in a short README or notebook conclusion.

MAINTAINED REFERENCE MAP

Use first-party documentation for version-sensitive details. Record the consulted version and date in each project.

Python tutorial	https://docs.python.org/3/tutorial/
NumPy user guide	https://numpy.org/doc/stable/user/index.html
pandas user guide	https://pandas.pydata.org/docs/user_guide/
Matplotlib user guide	https://matplotlib.org/stable/users/index.html
seaborn documentation	https://seaborn.pydata.org/
scikit-learn user guide	https://scikit-learn.org/stable/user_guide.html
PyTorch documentation	https://docs.pytorch.org/docs/stable/
TensorFlow Keras guide	https://www.tensorflow.org/guide/keras
Hugging Face Transformers	https://huggingface.co/docs/transformers/index
DVC documentation	https://dvc.org/doc
ONNX introduction	https://onnx.ai/onnx/intro/
FastAPI tutorial	https://fastapi.tiangolo.com/tutorial/
MLflow documentation	https://mlflow.org/docs/latest/
OpenAI developer quickstart	https://platform.openai.com/docs/quickstart
Gemini API reference	https://ai.google.dev/api
Claude API overview	https://platform.claude.com/docs/en/api/overview